

Separation of Policy and Mechanism

1 Core Idea

Definition

Mechanism is the low-level machinery that makes paging work. **Policy** is the decision-making rule that chooses what should happen.

In memory management, separating policy from mechanism means keeping the basic page-fault machinery in the kernel, while moving many higher-level decisions to a separate user-level component.

Main Principle

The kernel provides the mechanism for mapping pages; an external pager can decide the policy for where pages come from, where they go, and sometimes which pages should be replaced.

2 Three Main Parts

Part	Job	Location
MMU handler	Handles machine-dependent MMU details and installs mappings.	Kernel
Page-fault handler	Detects the fault, identifies the needed page, and coordinates the repair.	Kernel
External pager	Manages backing store and paging policy.	User space

3 Why Split Them?

Paging code can become complex because it mixes hardware details, kernel control flow, disk backing store, and replacement decisions. Splitting these concerns makes the system easier to change.

Concern	Best place for it
Hardware-specific translation	MMU handler.
Trap handling and process restart	Kernel page-fault handler.
Backing-store layout and paging policy	External pager.

4 Page Fault Flow

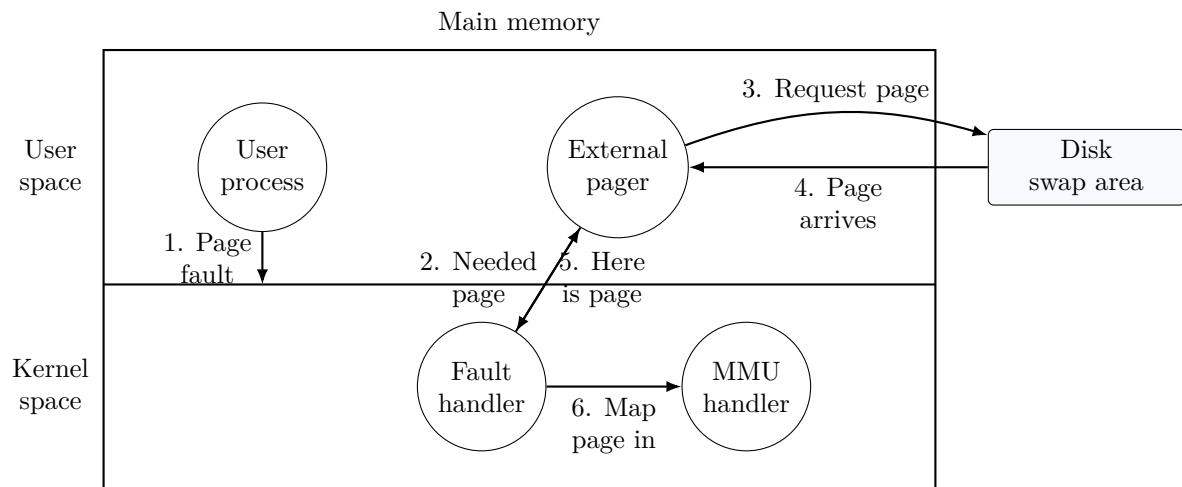


Diagram Meaning

The page fault starts in the kernel, but the needed page may be located and supplied by a user-level external pager.

5 Step-by-Step Explanation

1. The user process references a page that is not currently mapped.
2. The kernel page-fault handler determines which virtual page is needed.
3. The fault handler sends a request to the external pager.
4. The external pager reads the needed page from disk into its own address space.
5. The external pager tells the kernel where the page is.
6. The kernel asks the MMU handler to map that page into the faulting process at the correct virtual address.

After the mapping is installed, the faulting instruction can be restarted.

6 Process Creation and Mapping

When a process starts, the external pager may be notified so it can:

- create or initialize the process page map;
- allocate backing-store space if needed;
- handle future mappings of files, objects, or memory regions.

This keeps higher-level memory decisions outside the low-level kernel mechanism.

7 Where Page Replacement Fits

The clean design would put page replacement policy in the external pager. In practice, this is difficult because many replacement algorithms need hardware-maintained page bits.

Problem	Explanation
Referenced bit R	Shows whether a page was recently read or written.
Modified bit M	Shows whether the page is dirty and must be written back.
External pager limitation	A user-level pager may not directly see all R and M bits.

Because of this, the replacement algorithm may remain in the kernel, while the external pager handles backing-store operations.

8 Advantages and Disadvantages

Side	Explanation
Advantage	More modular code; easier to change paging policy; better separation between hardware details and high-level decisions.
Disadvantage	Extra messages and user-kernel crossings add overhead during page-fault handling.

Tradeoff

This design improves flexibility and modularity, but page faults may become slower because more components communicate during fault handling.

9 Final Summary

One-Box Summary

Mechanism: kernel code that traps, maps, and restarts.
Policy: decisions about paging and backing store.
External pager: user-level process that can manage policy.
Main tradeoff: modularity and flexibility versus extra overhead.